

Project Reflection – Digital Dice

ECE 1100

Om Patel

November 11th, 2025

In this Discovery Project, I was challenged to identify a real-world problem and engineer a complex solution using the ECE skills I learned over the semester. My starting idea stemmed from a common social issue I encounter in group settings at Georgia Tech: decision paralysis. Whether deciding on a place to eat or setting a meeting spot for a group project, we often resorted to flipping a coin or using online tools. To solve this, I wanted to create a "Digital Dice"—a physical device capable of generating a random number from one to six, mimicking the function of a traditional die.

The progress of this project was not straightforward and required me to completely change my design strategy. My initial design, a hardware-focused approach, involved building the logic entirely from scratch using discrete integrated circuits. Using software called TinkerCAD, I virtually created a design where a 555 timer continuously pulsed into a 4-bit binary counter (limited to count up to 6). The counter's output fed into a 7-segment display decoder to visualize the binary numbers on a screen. The system was controlled by a button, debounced using a Schmitt trigger and a D flip-flop. However, when I went to physically build this board at the ECE makerspace, "The Hive," I hit a major roadblock: there were no 4-bit binary counters available. While I considered using up/down counters, I realized this would require excessive space on the breadboard and complicate the wiring significantly.

This failure forced me to reevaluate and pivot to a software approach. I decided to use an Arduino Uno R3, leveraging a microcontroller to handle the logic I had previously attempted to build with hardware gates. I utilized TinkerCAD's Arduino simulation software to create a new blueprint, connecting the breadboard and 7-segment display directly to the Arduino. This phase required me to expand my skillset significantly; relying on my basic knowledge of Python and C, I taught myself the necessary C++ syntax to program the Arduino. I implemented code that utilized `randomSeed(analogRead(0))` to ensure true randomness and created a logic loop that would "freeze" the number when the button was pressed.

Through this process, I gained specific ECE skills that bridge the gap between hardware and software. I learned how to interface a microcontroller with external peripherals, such as a button and a 7-segment display. I also refined my TinkerCAD skills and learned the basics of C++. Ultimately, this project was a valuable lesson in adaptability. While the failure of the initial hardware design was frustrating, it pushed me to explore embedded systems.